

Gjennomgang av grunnmuren

En forklaring av alt som er bygget i repoet — forumet, sikkerhetsnettet og Fabrikken — skrevet for den som ikke har kjennskap til Laravel fra før.

Til utskrift og lesing. Ta den i det tempoet du vil.

August 2026 · Pull request #1 · github.com/jomoskus/katolskforum

Innhold

Slik brukes heftet	3
Del 1. Det store bildet	4
Del 2. Grunnbegrepene	5
Git, commit, gren og pull request	5
PHP, Composer og Laravel	5
Flyten i en Laravel-app	5
Database, migrasjoner og testdata	6
Del 3. Selve forumet	7
Datamodellen	7
La oss følge et svar	7
Fire valg som er verdt å forstå	7
Innlogging og sikkerhet	8
Del 4. Sikkerhetsnettet — portene	9
Del 5. Kanttilfellene — seks lag	11
Del 6. Fabrikken — prosessen	12
Del 7. Forståelsesporten	14
Del 8. Agent-infrastruktur og overvåking	16
Del 9. Kart over repoet	17
Del 10. Dine neste steg	18

Slik brukes heftet

Dette er den samme gjennomgangen som ble skrevet i Cursor, redigert for papir. Du trenger ikke å lese alt på én gang. Del 2 er ordboken. Del 3–5 er selve produktet. Del 6–8 er hvordan vi bygger videre uten at du mister kontrollen.

Tekst i `slik form` er filnavn, kommandoer eller klassenavn. Du trenger ikke å huske dem — poenget er at du kjenner dem igjen når du ser dem i en pull request.

Alt arbeidet ligger i pull request **#1** på grenen `cursor/laravel-forum-fabrikken`. Når du har forstått den på nivå 3 (se del 7) og flettet den inn, *er* dette repoet.

Del 1. Det store bildet

Det finnes nå én pull request som inneholder alt. Når du fletter den inn, har repoet ditt tre ting:

1. **Et ferdig forum.** Nettsiden katolskforum.no kan i praksis lanseres: forside med tråder, trådvisning med svar, innlogging med e-postbekreftelse, totrinnsverifisering og passkeys, alt på norsk.
2. **Et sikkerhetsnett.** Automatiske kontroller som all fremtidig kode må passere, satt på strengeste nivå.
3. **En prosess.** Fabrikken: skrevne spilleregler pluss automatikk som styrer hvordan idéer blir til kode, hvem (mennesker og KI-modeller) som kontrollerer hva, og hva *du* må forstå før noe slippes inn.

Hvorfor tre ting, ikke bare forumet?

Nesten all fremtidig kode vil skrives av KI-agenter. Da er sikkerhetsnettet og prosessen like viktige som selve produktet. Uten dem blir du et gummistempel, og forståelsesgjelden vokser i det stille.

Del 2. Grunnbegrepene

Hoppes over det du kan. Resten av heftet forutsetter at disse ordene betyr noe.

Git, commit, gren og pull request

Git er versjonskontroll: hele historikken til koden lagres som en kjede av **commits** — lagringspunkter med beskrivelse, som kapitler i en bok som bare kan få nye sider, aldri omskrevne gamle. En **gren** (branch) er en parallell arbeidslinje. Man jobber på en gren, og når arbeidet er ferdig ber man om å få det flettet inn i hovedlinjen (**main**) via en **pull request (PR)**: en forespørsel som viser nøyaktig hva som endres, der man kan diskutere, kjøre kontroller og godkjenne før fletting.

Alt vårt arbeid ligger på grenen `cursor/laravel-forum-fabrikken`. Fabrikken krever at all fremtidig endring også går via PR — også når du er alene i repoet. Årsaken er praktisk: CI-portene, modell-reviewene og forståelseserklæringen din er alle festet til PR-flaten. Uten PR finnes det ingen arkivert begrunnelse, og neste agent kan ikke ta over.

PHP, Composer og Laravel

PHP er programmeringsspråket nettsiden er skrevet i — det samme som driver Wikipedia, WordPress og gamle rbkweb (phpBB). Det kjører på serveren og lager HTML som nettleseren viser.

Composer er PHPs pakkebehandler: et verktøy som henter og oppdaterer tredjepartskode («pakker»). Filen `composer.json` lister hva vi bruker; `composer.lock` låser eksakte versjoner så alle — du, en agent, en server — får identisk oppsett. Kommandoene vi bruker mest, er også definert der: `composer test`, `composer fix`, `composer dev`.

Laravel er det mest brukte PHP-rammeverket. Tenk på det som et ferdig fundament med innlogging, databasehåndtering, e-post og sikkerhet, slik at man bare skriver det som er unikt for sin app. Vi bruker nyeste versjon (13) og følger konvensjonene slavisk, fordi det både mennesker og KI-modeller kan aller best. Jo mer «vanlig Laravel», desto sjeldnere hallusinerer agentene.

Artisan er Laravels kommandolinjeverktøy. Når du ser `php artisan migrate` eller `php artisan test`, er det Artisan som kjører.

Flyten i en Laravel-app

Når noen åpner en side, skjer dette, alltid i samme rekkefølge:

```
Nettleser → Rute → Middleware → Form Request → Controller → Policy → Modell/database → Blade-visning → HTML tilbake
```

- **Rute** (`routes/web.php`): kartet fra nettadresse til kode. «GET /threads/{thread} → vis denne tråden.»
- **Middleware**: kontrollposter alle forespørsler passerer, for eksempel «er du innlogget?». Vår egen `AddRequestContext` stempler hver forespørsel med en sporings-ID.

- **Form Request:** validering av innsendte skjema («tittel er påkrevd, maks 150 tegn») med norske feilmeldinger.
- **Controller:** trafikkdirigenten. Den tar imot forespørselen, spør databasen og velger hvilken side som skal vises. Den skal være tynn og dum — forretningsregler bor andre steder.
- **Policy:** regelboka for tillatelser. «Bare forfatteren får redigere.»
- **Modell** (Eloquent): en PHP-klasse som representerer en databasetabell. `Thread`-modellen er `threads`-tabellen; `$thread->posts` henter alle svarene automatisk.
- **Blade-visning:** HTML-maler med små kodesnutter, som blir siden brukeren ser. Forumets egne visninger skriver norsk bokmål direkte i malen — vi har bevisst ikke et oversettingslag for eget innhold, fordi forumet bare skal være på norsk.

Hvis du kan følge denne flyten for én handling — for eksempel å svare i en tråd — kan du lese nesten all kode i dette repoet. Det er nivå 2 i forståelsesporten.

Database, migrasjoner og testdata

Vi bruker **SQLite**: én fil som er hele databasen. Null drift, null kostnad, og bytte til MySQL eller PostgreSQL senere er en konfigurasjonsendring, ikke en omskriving. Alle tabeller er definert med portable kolonnetyper av samme grunn.

Migrasjoner er oppskrifter på databaseendringer, skrevet som kode: «lag tabellen threads med disse kolonnene». De kjøres i rekkefølge og endres aldri i etterkant — ny endring betyr ny migrasjonsfil. Slik kan hvem som helst, eller en server, bygge en identisk database fra null med `php artisan migrate`.

En **factory** lager testdata («gi meg 25 tråder med tilfeldig innhold»). En **seeder** fyller den lokale databasen med norsk demoinnhold — velkomstråd, Kari og Ola Nordmann, en tråd fra en slettet bruker — så man ser et levende forum under utvikling, inkludert kanttilfellene.

Del 3. Selve forumet

Forumet er bevisst minimalt: les, start tråd, svar. Alt annet (kategorier, søk, varsler, reaksjoner) er utsatt til det har gått gjennom Fabrikken med egen spesifisering.

Datamodellen

Tre tabeller pluss det startpakken kom med for innlogging:

Tabell	Innhold
<code>users</code>	Medlemmene: navn, e-post, passord (kryptert), 2FA-hemmeligheter, <code>is_admin</code> -flagg.
<code>threads</code>	Trådene: tittel, slug (URL-vennlig navn), type (artikkel eller lenke), innhold, ev. URL, forfatter.
<code>posts</code>	Svarene: innhold, hvilken tråd, forfatter.

En tråd er enten en **artikkel** — en tekst forfatteren har skrevet selv — eller en **lenke** — en peker til noe utenfor forumet, med forfatterens egen kommentar. Dette skillet er selve produktidéen, og det er kodet som en enum (`ThreadKind`) med to verdier. Forsiden viser alle tråder sortert på *sist aktivitet*.

Autorisasjonsmatrisen — hvem som får gjøre hva — står i `docs/DOMENE.md` og er testdekket. Kortversjonen: alle kan lese. Innloggede med bekreftet e-post kan starte tråd og svare. Forfatteren kan redigere eget innhold og slette egen tråd så lenge ingen har svart. Administrator kan slette alt, men aldri omskrive andres tekst.

La oss følge et svar gjennom systemet

Dette er «følg et svar»-øvelsen fra pensumet. Hvis du kan gjenfortelle den, er du på nivå 2 for vanlig funksjonalitet.

Du står på en trådside og trykker «Publiser svar». Nettleseren sender en POST til `/threads/{tråd}/posts`. Ruten krever `auth` og `verified` — innlogget og bekreftet e-post — ellers blir du sendt til innloggingen eller e-postbekreftelsen. `StorePostRequest` validerer at innholdet finnes og er under 40 000 tegn. Feiler det, sendes du tilbake med norsk feilmelding («Innhold må fylles ut.»).

`PostController::store` lager svaret, kobler det til tråden og deg, lagrer, regner ut hvilken side i tråden svaret havner på (25 svar per side), og sender deg dit med et hopp direkte til ditt eget innlegg (`#post-123`).

I det svaret lagres, «touches» tråden — dens `updated_at`-tidsstempel oppdateres. Siden forsiden sorterer på akkurat det feltet, hopper tråden til topps. Det er hele «sist aktivitet»-mekanikken: null egen kolonne, null egen observer, bare en innebygd Laravel-funksjon. En bevisst enkelhet. Hvis noen senere «forbedrer» den med en egen `last_activity_at`-kolonne uten å oppdatere ADR-en, skal du stoppe dem.

Fire valg som er verdt å forstå

Alle fire har en ADR — en kort, skriftlig beslutning i `docs/beslutninger/`. ADR-er er der for at agenter ikke skal «forbedre» bort noe som ser rart ut, men er bevisst.

1. Selvhelbredende URL-er

En tråds adresse er `/threads/tanker-om-kirkearets-rytme-2` — lesbar tekst pluss ID. Oppslaget skjer på ID-en bakerst, så teksten kan være utdatert: endrer noen tittelen, videresendes gamle lenker automatisk (HTTP 308) til ny adresse.

Dette fjerner et helt kobbel av problemer på én gang: to tråder med lik tittel, tråd med tittelen «Create» som ellers ville kollidert med adressen til ny-tråd-skjemaet, titler med bare emoji, og døde lenker etter tittelendring. Prisen er at URL-ene får et id-suffiks, som hos Discourse og GitHub Issues. Akseptert.

2. Innhold overlever kontosletting

Sletter et medlem kontoen, forsvinner personopplysningene (navn, e-post), men trådene og svarene står igjen som «Slettet bruker». Ellers ville alle andres svar mistet sammenhengen. All kode som viser forfattere tåler derfor at forfatteren er borte — det er håndhevet av typene og testene. `is_admin` kan for øvrig aldri masse-tilordnes; flagget settes manuelt.

3. Trygg Markdown

Innlegg skrives i Markdown (**fet**, lister, lenker). All HTML brukere måtte prøve å smugle inn, strippes, og `javascript:`-lenker nektes. Det stopper XSS: at noen får kjørt sitt eget skript i andres nettleser — klassisk forumangrep. Én klasse (`app/Support/Markdown.php`) eier denne jobben, og all visning går gjennom komponenten `<x-markdown>`. Aldri skal brukertekst skrives ut med den usikre Blade-syntaksen som tolker HTML.

4. Admin kan slette, aldri omskrive

Moderasjon er å fjerne, ikke å endre andres ord. Og en forfatter kan bare slette egen tråd så lenge ingen har svart — svarene «tilhører» samtalen. Dette er bevisste nei, dokumentert i kanttilfelleregisteret og testdekket som 403-svar (forbudt) når feil person prøver.

Innlogging og sikkerhet

Autentiseringen skrev vi ikke selv. Den følger med Laravels offisielle startpakke (Fortify): registrering, innlogging, glemt passord, e-postbekreftelse, totrinnskode (2FA) og passkeys (Face ID, fingeravtrykk eller sikkerhetsnøkkel i stedet for passord). Alt er oversatt til norsk via en standard språkpakke (`laravel-lang`).

Startpakken kom til og med med herdede produksjonsregler: minst 12 tegn passord i produksjon, sjekk mot kjente lekkede passord (Have I Been Pwned), og sperre mot destruktive databasekommandoer i produksjon. Vi skrev tester som beviser grensene — at 11 tegn feiler og 12 går gjennom — fordi mutasjonstesting avslørte at de opprinnelige testene ikke ville merket om noen senket kravet til 11.

Én detalj som er lett å overse: `laravel-lang` oversetter feltet `body` til «kropp». Vi overstyrer det alltid til «innhold», ellers ville feilmeldingen lyde «Kropp må fylles ut.» Det står i ADR-0004, nettopp for at neste agent ikke skal «rydde bort» overstyringen.

Del 4. Sikkerhetsnett — portene

Alt kjøres samlet med `composer test`, og CI nekter fletting hvis noe er rødt. Ingen port kan svekkes for å bli grønn. Skal en terskel endres, krever det egen PR med skriftlig begrunnelse (ADR).

Tenk på portene som et kontrollsjekkpunkt en agent ikke kan snakke seg forbi. Tekst i `AGENTS.md` er veiledning. Portene er håndhevelse. Forskning på KI-kodebaser i 2026 er entydig på dette: det som kan sjekkes maskinelt, skal sjekkes maskinelt.

Port	Hva den fanger	Bilde
Pint	Formatering: innrykk, hermetegn, rekkefølge.	Korrekturleseren
Rector	Utdaterte mønstre og manglende typer. Kan autofikse.	Språkvaskeren
PHPStan (maks)	Logiske typefeil uten å kjøre koden: «her kan verdien være tom, men du behandler den som tekst».	Revisoren
Pest-testene	157 små programmer som bruker forumet og sjekker at alt oppfører seg riktig.	Testpanelet
Linjedekning $\geq 95\%$	At nesten hver kodelinje faktisk kjøres av minst én test.	Hvor mye av huset inspeksjonen har vært innom
Typedekning 100 %	At hver funksjon deklarerer hva den tar inn og gir ut.	At alle dører er merket
Mutasjonsscore $\geq 85\%$	At testene faktisk ville merket sabotasje. Se under.	Innbruddstesten
Arkitekturtester	At kodens struktur følger reglene.	Reguleringsplanen
<code>composer audit</code>	Kjente sikkerhetshull i tredjepartspakker.	Heftelsessjekken

Alt som kan autofikses, fikses med én kommando: `composer fix` (Rector + Pint). Agenter, du og CI bruker nøyaktig de samme kommandoene. Det er bevisst — uenige verktøykjeder er en av de vanligste årsakene til at agenter spinner.

Mutasjonstesting

Vanlig dekning måler bare om en linje ble *kjört* — ikke om noen ville *merket* at den ble ødelagt. Du kan ha 100 prosent linjedekning og likevel mangle tester som fanger at noen bytter «minst 12 tegn» til «minst 8».

Mutasjonstesteren lager derfor hundrevis av bevisste sabotasjer («mutanter»): bytter `>=` til `>`, fjerner en valideringsregel, snur en betingelse. Så kjøres testene mot hver sabotasje. Feiler minst én test, er mutanten «drept» — bra. Overlever den, har du bevis for et hull i testene.

Under byggingen tvang overlevende mutanter fram tester som ellers ikke ville blitt skrevet: at passordkravet skiller nøyaktig 11 fra 12 tegn, at svar nummer 25 blir på side 1 mens nummer 26 havner på side 2, at de norske feilmeldingene faktisk er norske. Vi endte på 100 prosent — alle 161 mutanter drept. Porten krever minst 85 prosent for all fremtidig kode, så det er slingringsmonn uten at nivået kan skli.

Hvorfor dette er «dagens konsept» i PR #1.

Hvis du bare husker én teknisk idé fra grunnmuren, så husk denne: dekning sier at testen besøkte rommet. Mutasjonsscore sier at testen ville oppdaget at møblene var byttet ut.

Del 5. Kanttilfellene — seks lag

Ønsket var å fange kanttilfeller tidlig, på mange måter. Strategien i `docs/KANTTILFELLER.md` har seks lag, fra tanke til drift:

1. **Før koden skrives.** Hver spesifikasjon må ha en kanttilfelle-seksjon, og minst to *forskjellige* modellfamilier genererer tilfeller hver for seg. Ulike modeller overser ulikt.
2. **I testene.** Kanttilfellene blir tester med grenseverdier: $n-1$, n , $n+1$. 11 tegn / 12 tegn. 24 svar / 25 svar / 26 svar.
3. **Mutasjonstesting.** Finner hullene ingen tenkte på.
4. **Statisk analyse og strenge typer.** Fanger null- og typefeil før koden i det hele tatt kjøres. PHPStan på maksnivå avslørte underveis at et API-kall agenten antok fantes, ikke fantes i det hele tatt — akkurat den typen hallusinasjon flerlagsporter skal fange.
5. **I hverdagen.** Demoinnholdet inneholder kanttilfeller (slettet bruker, lenketråd, tom tilstand) så man *ser* dem daglig, ikke bare i tester.
6. **I produksjon.** Overvåking fanger det som slapp gjennom, og hver reell feil blir regresjonstest pluss oppføring i registeret. Det er du som formulerer oppføringen. Å beskrive feilen er å forstå den.

Registeret er allerede fylt med rundt tretti håndterte tilfeller, med status og henvisning til testen som beviser håndteringen. Pluss en sjekkliste for alle fremtidige spesifikasjoner: tomt og enormt, unicode og æøå, tid og klokkeskjevhets, tilgang, samtidighet, livssyklus.

Noen åpne tilfeller er bevisst utsatt, merket : dobbeltklikk på «Publiser svar», spam-registreringer, og kappløp der noen sletter kontoen mens en annen leser siden. De kommer tilbake som issues når forumet nærmer seg åpning.

Del 6. Fabrikken — prosessen

Grunnidéen: den som skriver koden skal aldri være den eneste som vurderer den. Maskiner avgjør alt maskiner kan avgjøre, så modellene og mennesket kan bruke kreftene på intensjon og design.

Stegene

Steg	Hvem	Hva
0. Idé	Du	Et GitHub-issue. Tre setninger kan være nok.
1. Spesifikasjon	GLM	Utkast: mål, akseptansekriterier, kantilfeller, testplan, avgrensninger.
2. Kritikk	Kimi + GPT-OSS	To andre modellfamilier angriper utkastet. Du godkjenner konsolidert spesifikasjon.
3. Plan	Implementasjonsagent	Hvilke filer, migrasjoner, tester, risiko.
4. Implementasjon	Cursor-skyagent	Egen gren, testdrevet, alt grønt før PR.
5. Portene	CI + tre familier	Maskinelle porter, deretter Codex-review og Droid med GLM og Kimi.
6. Forståelsen	Du	Du må forstå endringen på definert nivå. Se del 7.
7. Fletting	Du	Bare mennesket fletter. Squash-merge.
8. Etterpå	Overvåking + du	Feil blir tester og registeroppføringer.

Verktøykartet — faste abonnementer

Bugbot fra Cursor er bevisst utelatt. Den prises per bruk. Alt under dekkes av faste abonnementer.

Abonnement	Rolle
Codex (200 USD)	Automatisk PR-review (inkludert). Spesifikasjon og implementasjon via @codex på issue eller PR.
Cursor (200 USD)	Implementasjonsagenter i editor, CLI og sky. Manuell review av store eller risikable endringer.
Synthetic (30 USD)	GLM, Kimi og GPT-OSS — tre modellfamilier til utkast, kritikk og to automatiske PR-reviews. Fast pris.

Automatiseringen: etiketter er knappene

GitHub er tilstandsmaskinen. Du setter en etikett på et issue; workflowen `fabrikken-dispatch` fyrer av riktig agent, poster resultatet som kommentar og fjerner etiketten. Sett den på igjen for å kjøre steget på nytt.

Etikett	Hva skjer	Kostnad
<code>fabrikk:spesifikasjon</code>	GLM skriver spesifikasjonsutkast som kommentar.	Fast pris (Synthetic)
<code>fabrikk:kritikk</code>	Kimi og GPT-OSS kritiserer utkastet hver for seg.	Fast pris (Synthetic)
<code>fabrikk:implementer</code>	En Cursor-skyagent lanseres, implementerer og åpner PR.	Inkludert bruk først; stoppes av taket ditt

Når PR-en åpnes, kjører tre ting automatisk: alle kvalitetsporter (`ci`), to Droid-reviews (GLM og Kimi), og sjekken som krever at PR-beskrivelsen har «Til mennesket»-seksjonen (`fabrikk-lint`). Codex reviewer via sin GitHub-app, når du har slått den på.

Promptene som styrer hvert steg ligger i `.github/fabrikken/` og versjoneres som kode. Spesifikasjonsforfatteren (GLM) er bevisst en annen modellfamilie enn kritikerne (Kimi og GPT-OSS). Ved uenighet mellom modeller oppsummerer en tredje familie, og du avgjør.

To tekniske funn som styrer oppsettet.

Mentions som `@codex` og `@cursor` må komme fra et menneske — bot-postede mentions trigger ikke agenter. Derfor bruker workflowen CLI og API direkte. Og Cursor-skyagenter krever at on-demand er slått på, men de trekker fra planens inkluderte bruk *først*. Taket du setter, for eksempel 10–20 USD, er bare et sikkerhetsnett.

Det manuelle sporet består: kommenter `@codex` eller `@cursor` selv på et issue. Cursor har i tillegg native Automations på cursor.com/automations for hendelses- og tidsstyrte agenter uten egen workflow-kode.

Del 7. Forståelsesporten

Ingenting flettes før mennesket forstår det, i definert grad. Godkjenningen din i GitHub *er* forståelseserklæringen, og den er først gyldig når kravene er oppfylt.

Nivåene

Nivå	Du kan, uten hjelp ...	Kreves for
1 Bruker	si med egne ord hva endringen gjør og hvorfor	alt
2 Kartleser	peke på hvor i koden det skjer, og hvilke tester som beviser det	vanlig funksjonalitet og UI
3 Navigator	forklare hvorfor slik, hva som ble valgt bort, kanttilfellene — og bestå kontrollspørsmål	datamodell, sikkerhet, personvern, porter, ADR-er
4 Mester	endre eller reversere selv, uten agent	aldri et flettekrav — et mål å vokse mot

Er du i tvil om typen, velger du det høyeste aktuelle nivået. PR #1 er selv nivå 3: den fastlegger datamodell, sikkerhetsregler og selve prosessen.

Slik oppfylles nivåene

Agenten forbereder. Hver PR må avsluttes med en «Til mennesket»-seksjon på norsk: jargongfri forklaring, foreslått nivå, omvisning i filene i leserekkefølge, tre kontrollspørsmål ved nivå 3, og «dagens konsept» — én ting å lære per PR. Dette er påbudt i `AGENTS.md`, og `fabrikk-lint` i CI nekter PR-er som mangler seksjonen. Porten kan ikke glemmes.

Du beviser, skriftlig i PR-en:

- **Nivå 1** (under ett minutt): én egenformulert setning. Ikke kopier agentens tekst.
- **Nivå 2** (noen minutter): følg omvisningen, skriv 3–5 setninger teach-back. Ved UI-endringer: prøv funksjonen selv.
- **Nivå 3** (et kvarter): svar på kontrollspørsmålene uten å slå opp først. En modell fra en *annen* familie enn den som skrev koden, verifiserer svarene mot diffen og påpeker hull. Hull betyr: les mer, svar på nytt.

Tidsbruken er bevisst. Det er prisen for å forbli sjef i eget repo.

Pensum

Nå — for å styre trygt fra dag én

- Kjøre forumet lokalt: `composer setup`, `composer dev`, `php artisan migrate:fresh --seed`.
- Lese resultatet av `composer test` og vite hva hver port sjekker — og hvorfor ingen kan svekkes for å bli grønn.
- PR-flyten: gren → PR → porter → reviews → forståelsesport → squash-merge.

- Domenereglene i `docs/DOMENE.md`, særlig autorisasjonsmatrisen.
- Hva som aldri committes (`.env`, hemmeligheter) og hvor secrets bor (GitHub Actions-secrets).
- Hvordan be en agent om review, verifisering av teach-back, eller en forklaring på et hvilket som helst nivå.

Snart — for å nå nivå 2 rutinemessig

- Laravel-flyten: rute → Form Request → controller → policy → modell → visning.
- Migrasjoner: hvorfor de er append-only, og hva `php artisan migrate` gjør.
- Relasjonene vi bruker, og at svar «touches» tråden.
- Øvelsen: start i `routes/web.php` på `threads.posts.store` og følg et svar til testen som beviser det.

Etter hvert — for nivå 3 på alt

- Validering i dybden, policies og Gates.
- Lese en overlevende mutant og skrive testen som dreper den.
- Tolke PHPStan-feil på maksnivå.
- Fortify-flytene: registrering, verifisering, 2FA, passkeys.

Laravel Bootcamp (bootcamp.laravel.com) er gratis og nærmest pensum for nivå 2. Dette repoet er selv pensum: les `docs/ARKITEKTUR.md`, så koden den peker på.

Læringsrutiner

- **Dagens konsept.** Hver PR forklarer ett konsept fra diffen. Over tid dekker konseptene pensumet uten egne studieøkter.
- **Prod-feil skrives av mennesket.** Når en produksjonsfeil føres inn i kanttilfelleregisteret, er det du som formulerer oppføringen.
- **Månedlig stikkprøve.** Velg en tilfeldig flettet PR og forklar den for deg selv på nivået den krevde. Klarer du ikke, har forståelsesgjelden begynt å vokse — juster tempoet.

Del 8. Agent-infrastruktur og overvåking

Det agentene leser og bruker

`AGENTS.md` i roten er instruksene alle KI-verktøy leser automatisk: kommandoer, regler, grenser. «Aldri svekke en port.» «All markdown gjennom den trygge rendereren.» «Innhold overlever kontosletting.» Filen er bevisst kort. Forskning viser at lange, genererte instruksjonsfiler gjør agenter dårligere, ikke bedre. Det som kan utledes av koden, står ikke der. Det som er ikke-innlysende, står der.

Laravel Boost er Laravels offisielle MCP-server. MCP er standardprotokollen som gir KI-verktøy tilgang til systemer. Via Boost kan agenter slå opp i *riktig versjon* av Laravel-dokumentasjonen, inspisere databasen og rutene, og lese loggene — i stedet for å gjette. Offisielle ferdigheter (skills) for Livewire, Fortify, Pest og Tailwind ligger i `.cursor/skills/`.

`.cursor/environment.json` gjør at Cursor-skyagenter — virtuelle maskiner i skyen som jobber selvstendig — automatisk får installert PHP og hele appen når de våkner, via `.cursor/install.sh`.

Overvåking i produksjon

Sentry er en feilsparingstjeneste. Gratisnivået holder lenge for et forum. Den aktiveres av én miljøvariabel (`SENTRY_LARAVEL_DSN`) og er avslått lokalt og i tester. Alle uhåndterte feil rapporteres med rute og bruker-ID — aldri e-post eller navn.

Hver forespørsel stemples med en unik ID som følger med i alle logger og i kjøbber, og returneres til nettleseren som headeren `X-Request-Id`. Melder en bruker om feil, kan ID-en slås opp og hele hendelsesforløpet finnes igjen. En innkommende ID gjenbrukes bare hvis den er en gyldig UUID — ellers erstattes den, så ingen kan smugle loginjeksjon inn via headeren.

Rutinen står i `docs/OBSERVABILITET.md`: feil → gjenskap som test → fiks via Fabrikken → inn i kanttilfelleregistret. Ukentlig triage av nye Sentry-grupper. En ekstern ping (for eksempel UptimeRobot, gratis) mot forsiden og helsesjekken `/up`.

Før lansering må `MAIL_MAILER=log` byttes til en ekte e-postleverandør. E-postbekreftelse avhenger av det. Resend og Postmark har rimelige nivåer. `APP_DEBUG` skal være avslått i produksjon.

Del 9. Kart over repoet

Du trenger ikke å huske dette. Det er et kart du slår opp i når en PR peker på en fil.

AGENTS.md	instruks for KI-agenter
README.md	inngangsdør med kommandoer
composer.json	avhengigheter og kommandoene (test, fix, ...)
routes/web.php	alle nettadressene
app/Models/	Thread, Post, User (databasen som kode)
app/Http/Controllers/	trafikkdirigertene
app/Http/Requests/	skjemavalidering med norske meldinger
app/Policies/	hvem får gjøre hva
app/Support/Markdown.php	trygg tekst-rendering
resources/views/	HTML-malene (norsk UI)
database/migrations/	databaseoppskriftene
database/seeds	demoinnhold
database/factories	testdata-oppskrifter
tests/	157 tester, inkl. kanttilfellene
docs/	prosess, pensum, kanttilfeller, ADR-er
.github/workflows/	ci, droid-review, fabrikk-dispatch, fabrikk-lint
.github/fabrikk/	promptene for stegene
.cursor/	MCP, skills og skyagent-miljø

Dokumentene i docs/ er skrevet for mennesker, på norsk:

Dokument	Innhold
FABRIKKEN.md	Proessen, automatiseringen, engangsoppsettet.
FORSTAAELSE.md	Nivåene, mekanismene, pensumet.
KANTTILFELLER.md	Seks lag og det levende registeret.
DOMENE.md	Ordbok og autorisasjonsmatrise.
ARKITEKTUR.md	Struktur og viktige mekanismer.
OBSERVABILITET.md	Sentry, logger, rutiner.
beslutninger/	ADR-0001 til 0007.
gjennomgang.pdf	Dette heftet, til utskrift.

Del 10. Dine neste steg

1. Engangsoppsettet i slutten av `docs/FABRIKKEN.md`, i denne rekkefølgen:

- Installer Codex-GitHub-appen og slå på automatisk kodegjennomgang.
- Legg inn GitHub-secrets: `FACTORY_API_KEY`, `SYNTHETIC_API_KEY`, `CURSOR_API_KEY`.
- Slå på on-demand for Cursor-skyagenter med et lavt tak (10–20 USD).
- Opprett de tre etikettene `fabrikk:spesifikasjon`, `fabrikk:kritikk` og `fabrikk:implementer`. Ferdige kommandoer står i `FABRIKKEN.md`.
- Sett grenvern på `main`: krev at `ci` / `Kvalitetsporter` og `fabrikk-lint` / `PR` følger `Fabrikk` er grønne, krev én godkjenning, slå på `squash-merge`.

2. Forståelsesporten på PR #1. Kjør `composer setup` & `composer dev` lokalt og klikk rundt. Følg omvisningen i PR-beskrivelsen (ni filer i leserekkefølge). Skriv teach-back. Svar på de tre kontrollspørsmålene i en kommentar:

- Hva må være sant om en bruker før hen kan skrive et innlegg — og hvor i koden håndheves det?
- Hva skjer med trådene og svarene til en som sletter kontoen sin, og hvorfor valgte vi akkurat det?
- Mutasjonsscoren er 100 prosent — hva betyr det, og hvorfor er ikke høy linjedekning alene godt nok?

Be deretter en agent fra en annen modellfamilie enn den som skrev koden om å verifisere svarene. Når det er grønt: flett.

3. Før lansering — egne Fabrikk-issues, ikke noe som skal smyges inn:

- Ekte e-postleverandør (verifiseringen krever det).
- `SENTRY_LARAVEL_DSN` i produksjon.
- Server, HTTPS og DNS mot `katolskforum.no`.
- Backup av SQLite-filen (Litestream eller enkel kopi).

Om noe i denne gjennomgangen fortsatt er uklart, er det i seg selv et nyttig signal. Si hvilken del, så kan den utdypes — eller gjøres om til en oppføring i pensumet.

Skrevet august 2026 som papirutgave av gjennomgangen i Cursor-samtalen som bygde grunnmuren. Kilden er `docs/gjennomgang.html`. Regenereres med `weasyprint docs/gjennomgang.html docs/gjennomgang.pdf`.